Nama: Afrizal Rafli Kusuma Wardana
Kelas: TI 3C
Repository:
https://github.com/rizalrfli/24417020007-mobile-course/tree/main/week5_offline

→ Screenshot tersedia di dalam github

> **Pertanyaan Praktikum 1**
>
> 1. Mengapa SharedPreferences.getInstance() tidak boleh dipanggil di dalam method build() widget?
> 2. Jelaskan alur data dari saat switch ditekan sampai nilai tersimpan di disk dan tema berubah.
>   a. Apa kelebihan dan risiko pendekatan *optimistic update* pada toggle()?

Jawaban:

1. Method build() dapat dipanggil berkali-kali ketika widget mengalami rebuild. Jika SharedPreferences.getInstance() dipanggil langsung di dalam build(), proses pembacaan preferensi dapat dilakukan berulang kali sehingga tidak efisien dan berpotensi menyebabkan tampilan berkedip atau tidak konsisten.
2. Switch Tema → toggle() → state berubah → UI rebuild → repository → SharedPreferences → disk. Ketika pengguna menekan switch, method toggle() pada DarkModeNotifier dipanggil. Nilai tema sebelumnya dibaca kemudian dibalik menjadi nilai baru. State langsung diubah menggunakan AsyncData(next) sehingga MaterialApp yang melakukan watch terhadap darkModeProvider akan rebuild dan tema berubah. Setelah itu repository menjalankan setDarkMode(next) untuk menyimpan nilai bool ke SharedPreferences. SharedPreferences kemudian menyimpan nilai tersebut ke penyimpanan lokal perangkat. Ketika aplikasi dibuka kembali, DarkModeNotifier.build() membaca kembali nilai tersebut sehingga tema sebelumnya tetap digunakan. Modul_Praktikum_Minggu5_Local_S…
3. Kelebihannya adalah UI terasa lebih cepat dan responsif karena perubahan tema dilakukan terlebih dahulu tanpa harus menunggu proses penyimpanan selesai.

Pertanyaan Praktikum 2

1. Mengapa kolom dirty bertipe INTEGER dan bukan BOOLEAN?
2. Apa fungsi parameter openDb pada constructor NoteRepository?
3. Mengapa query memakai where: 'id = ?' dan whereArgs, bukan interpolasi string?
4. Apa yang terjadi jika Anda menambah kolom baru di onCreate tanpa menaikkan version?

Jawaban:

1. SQLite tidak mempunyai tipe data BOOLEAN asli seperti Dart. Karena itu nilai boolean disimpan sebagai INTEGER:

- 1 = true
- 0 = false

   Pada model Note, proses konversi dilakukan melalui toMap() dan fromMap(). Ketika data disimpan, true diubah menjadi 1. Ketika data dibaca kembali, nilai 1 diubah menjadi true.

2. Parameter openDb digunakan untuk menerapkan **dependency injection**. Dengan cara ini repository tidak selalu terikat langsung dengan database SQLite asli.
3. Cara ini lebih aman karena mencegah **SQL Injection** dan menghindari masalah ketika nilai mengandung karakter khusus seperti tanda kutip. Database akan menangani parameter tersebut secara aman.
4. onCreate hanya dijalankan ketika file database pertama kali dibuat. Jika database sudah ada kemudian kita hanya menambahkan kolom baru di onCreate, kode tersebut tidak akan dijalankan lagi.

Pertanyaan Praktikum 3

1. Mengapa setelah setiap mutasi perlu meng-invalidate notesProvider **dan** dirtyCountProvider? Apa yang terjadi jika hanya salah satu?
2. Bagaimana cara Anda memicu state **error** secara sengaja untuk menguji tampilan _ErrorView?
3. Mengapa aplikasi tetap berfungsi dalam mode pesawat walaupun tidak ada kode khusus untuk mode offline?

Jawaban:

1. Kedua provider membaca sumber data yang sama tetapi digunakan untuk kebutuhan yang berbeda. notesProvider digunakan untuk menampilkan daftar catatan, sedangkan dirtyCountProvider digunakan untuk menampilkan jumlah catatan yang belum tersinkronisasi. Karena itu setelah tambah, edit, atau hapus data, keduanya perlu di-invalidate agar membaca ulang data terbaru. Jika hanya notesProvider yang di-invalidate, daftar catatan sudah berubah tetapi jumlah badge dirty dapat tetap memakai nilai lama. Sebaliknya, jika hanya dirtyCountProvider yang di-invalidate, badge sudah benar tetapi daftar catatan dapat tetap menampilkan data lama.
2. Salah satu cara yang paling aman adalah mengganti noteRepositoryProvider dengan repository palsu yang sengaja melempar exception.
3. Karena seluruh operasi CRUD catatan dilakukan terhadap **SQLite lokal** di perangkat, bukan server internet.

Tabel Uji:

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	Empty state	Jalankan aplikasi pertama kali	Ikon + teks "Belum ada catatan"	Ikon + teks "Belum ada catatan"	Lulus
2	Create	Tekan + Catatan, isi judul, Simpan	Catatan muncul paling atas, ikon awan oranye, badge = 1	Catatan muncul paling atas, ikon awan oranye, badge = 1	Lulus
3	Validasi	Simpan dengan judul kosong	Muncul "Judul wajib diisi", dialog tidak tertutup	Muncul "Judul wajib diisi", dialog tidak tertutup	Lulus
4	Update	Ketuk catatan, ubah isi, Simpan	Isi berubah, catatan naik ke posisi teratas	Isi berubah, catatan naik ke posisi teratas	Lulus
5	Delete	Tekan ikon tempat sampah	Catatan hilang, badge berkurang bila catatan itu dirty	Catatan hilang, badge berkurang bila catatan itu dirty	Lulus
6	Persistensi	Tutup aplikasi total, buka lagi	Semua catatan masih ada	Semua catatan masih ada	Lulus
7	Mode pesawat	Aktifkan mode pesawat, ulangi skenario 2, 4, 5	Semua tetap jalan tanpa error	Semua tetap jalan tanpa error	Lulus

Pertanyaan Praktikum 4

1. Apa perbedaan cache-first dan network-first? Berikan satu contoh data yang lebih cocok memakai network-first.
2. Jelaskan skenario kehilangan data yang dapat terjadi akibat markAllSynced(), lalu usulkan perbaikannya.
3. Mengapa diperlukan saklar forceOffline padahal sudah ada mode pesawat?
4. Mengapa fetchAndCache() menulis cache di dalam transaksi?

Jawaban:

1. **Cache-first** berarti aplikasi membaca data lokal terlebih dahulu supaya informasi dapat langsung ditampilkan. Setelah itu data terbaru diambil dari jaringan dan cache diperbarui. Strategi ini cocok untuk data yang tidak harus selalu real-time, misalnya artikel, daftar berita yang sudah pernah dibaca, katalog, atau konten informasi.
2. Masalah dapat terjadi ketika proses sinkronisasi sedang berlangsung lalu pengguna kembali mengedit suatu catatan. Misalnya catatan A dengan dirty = 1 mulai dikirim ke server. Saat upload masih berjalan, pengguna mengubah catatan A lagi. Versi baru tersebut juga menjadi dirty.
3. Jika bergantung pada mode pesawat atau Wi-Fi perangkat, hasil testing dapat berbeda-beda sesuai kondisi jaringan. Dengan forceOffline, aplikasi dapat mensimulasikan kondisi offline kapan saja tanpa mengubah pengaturan perangkat.
4. Transaksi memastikan proses menghapus cache lama dan memasukkan cache baru dilakukan sebagai **satu operasi atomik**. Jika seluruh proses berhasil, cache baru akan digunakan. Jika terjadi kegagalan di tengah proses, transaksi dapat dibatalkan sehingga cache lama tetap utuh. Tanpa transaksi, kegagalan setelah beberapa data dimasukkan dapat menghasilkan cache yang hanya terisi sebagian atau tidak konsisten.

Tabel Uji:

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	Isi cache	Saat **online**, buka halaman Posts (ikon artikel)	100 posts tampil, tersimpan di `cached_posts`	100 posts tampil,	Lulus

				tersimpan di `cached_posts`	
2	Cache saat offline	Aktifkan mode pesawat, tutup lalu buka aplikasi, buka Posts	Posts tetap tampil dari cache	Posts tetap tampil dari cache	Lulus
3	Antrean dirty	Masih offline, tambah 3 catatan	Badge menunjukkan angka 3	Badge menunjukkan angka 3	Lulus
4	Sync ditolak	Settings → aktifkan **Paksa mode offline**, tekan ikon Sinkronkan	Snackbar "Perangkat offline…", badge tetap 3	Snackbar "Perangkat offline…", badge tetap 3	Lulus
5	Sync berhasil	Matikan saklar offline, tekan Sinkronkan	Setelah ±1 detik snackbar "3 catatan berhasil disinkronkan", badge hilang, ikon awan hijau	Setelah ±1 detik snackbar "3 catatan berhasil disinkronkan", badge hilang, ikon awan hijau	Lulus
6	Refresh background	Online, buka Posts	Daftar tampil seketika dari cache lalu diperbarui	Daftar tampil seketika dari cache lalu diperbarui	Lulus

> **Pertanyaan Praktikum 5**
>
> 1. Mengapa kita menguji provider dengan ProviderContainer + overrideWithValue dan bukan dengan membuka database asli?
> 2. Apa manfaat parameter latency pada syncNotes bagi pengujian?
> 3. Tuliskan satu test tambahan yang menurut Anda penting namun belum ada, beserta alasannya.

Jawaban:

1. Karena tujuan test tersebut adalah menguji **logika provider**, bukan kemampuan SQLite. Dengan ProviderContainer dan overrideWithValue, kita dapat mengganti repository asli menggunakan FakeNoteRepository.
2. Pada aplikasi normal, latency dapat digunakan untuk mensimulasikan waktu pengiriman data ke server, misalnya 1 detik.

3.

```dart
test('update catatan membuat dirty menjadi true', () async {
  final oldNote = Note(
    id: 1,
    title: 'Catatan lama',
    body: '',
    updatedAt: DateTime(2026, 10, 5, 10),
    dirty: false,
  );

  final updated = oldNote.copyWith(
    title: 'Catatan baru',
    dirty: true,
  );

  expect(updated.dirty, isTrue);
  expect(updated.title, 'Catatan baru');
});
```

Test ini penting karena setiap perubahan yang dilakukan pengguna harus ditandai sebagai belum tersinkronisasi. Jika dirty tidak kembali menjadi true, perubahan terbaru bisa tidak pernah dikirim ke server.

Prompt

Kembangkan project praktikum (atau buat baru) dengan ketentuan berikut:

1. Preferensi: toggle tema gelap/terang + waktu terakhir dibuka melalui SharedPreferences.

2. CRUD catatan persisten melalui SQLite (sqflite) dengan repository lokal + Riverpod; daftar diurutkan updated_at terbaru.

3. Offline-first: cache-first untuk data bacaan, dirty flag + syncNotes untuk tulisan, dan aturan konflik eksplisit yang didokumentasikan.

4. Bukti mode pesawat: screenshot daftar catatan saat offline dan badge dirty sebelum/sesudah sync.

5. Testing: minimal 2 test lulus (1 unit test model + 1 test provider dengan repository palsu).

6. AI Challenge: dokumentasikan prompt, tabel perbandingan storage, keputusan final, serta alasan teknis di docs/.

7. Repository: push ke repository portfolio pada folder 05-week-5-local-storage-offline-first/ dengan struktur lib/, test/, docs/, README.md, dan screenshots/.

README minimal memuat: tujuan, fitur utama, stack teknologi, cara menjalankan, hasil yang dicapai, aturan konflik yang dipilih, dan temuan verifikasi AI.

Ringkasan output AI yang didokumentasikan

Hasil awal merekomendasikan SharedPreferences untuk tema dan waktu pembukaan, serta sqflite untuk catatan. Hive diposisikan sebagai alternatif key-value untuk objek sederhana, sedangkan Drift menawarkan query bertipe dan stream. Skema catatan menyertakan id, title, body, updated_at, serta dirty.

Dokumentasi awal juga memuat klaim bahwa counting dirty dengan indeks menjadi O(1), SharedPreferences bersifat dynamic, dan Hive pasti lambat untuk 1000 item. Klaim tersebut tidak saya pertahankan: getter preferensi memiliki tipe, counting tetap bergantung pada jumlah entri yang diperiksa, dan performa perlu benchmark yang sesuai beban aplikasi.

Prompt pengembangan dan hasil audit

docs/ai-challenge.md pada project akhir mencatat prompt pengembangan dengan tujuh ketentuan: preferensi persisten, CRUD sqflite dan Riverpod, cache-first dan dirty sync, bukti mode pesawat, minimal dua test, dokumentasi keputusan AI, serta struktur repository. Dokumen itu merupakan prompt pengembangan yang berbeda dari prompt perbandingan storage.

Pertanyaan Refleksi

1. **Mengapa daftar catatan tidak boleh disimpan di SharedPreferences? Apa yang rusak jika aturan ini dilanggar?**

   SharedPreferences cocok untuk key-value kecil. Catatan membutuhkan pengurutan, pencarian, update per baris, dirty flag, dan transaksi. Jika koleksi disimpan sebagai satu JSON, setiap edit harus membaca dan menulis ulang daftar. Dua operasi bisa saling menimpa dan perubahan tertunda sulit dipertahankan dengan aman. Menyimpan JSON secara teknis mungkin, tetapi membuat konsistensi catatan lebih rumit dan tidak sesuai dengan jaminan penyimpanan yang diperlukan.

2. **Kapan cache-first cukup dan kapan membutuhkan strategi lain seperti network-first?**

Cache-first cukup untuk bacaan atau materi yang tetap berguna meskipun belum paling baru. Saya memakai strategi ini pada Posts. Harga, stok saat checkout, dan status pembayaran memerlukan data segar; network-first dapat dipakai dengan timeout, label waktu cache, dan validasi server saat konfirmasi. Data yang terus berubah juga dapat memerlukan stream atau push. Pemilihan strategi mengikuti toleransi terhadap data yang tertinggal.

3. **Bagaimana dirty flag menjadi antrean sync tanpa memblokir UI? Kapan oubox perlu?**

   Mutasi disimpan ke SQLite dengan dirty = 1, lalu provider memperbarui UI. syncNotes membaca snapshot pending dan melakukan upload secara async. ACK membersihkan versi yang sama, sedangkan edit baru dan kegagalan jaringan tetap meninggalkan dirty. Tombstone mengikutkan penghapusan ke antrean. Outbox diperlukan jika setiap operasi, urutan, retry, dan idempotency key harus dipertahankan, misalnya pada audit atau transaksi. Perubahan data dan penambahan outbox harus berada dalam satu transaksi.

4. **Bagian mana dari rekomendasi AI yang ditolak dan mengapa?**

   Saya menolak pembersihan dirty hanya setelah delay karena proses itu belum membuktikan pengiriman ke server. Saya juga mengoreksi klaim cache SharedPreferences, counting indeks O(1), dan anggapan Hive pasti lambat untuk 1000 item. Keputusan harus cocok dengan kode, bukti, serta karakteristik storage. Drift tidak dipilih karena sqflite sudah sesuai kebutuhan dan ketentuan tugas, bukan karena Drift dianggap keliru.

Kesimpulan

Hal terpenting yang saya pelajari adalah penyimpanan lokal dan sinkronisasi harus menjaga versi data yang benar. SharedPreferences memenuhi kebutuhan preferensi, sedangkan SQLite menyimpan catatan dan cache melalui repository yang diamati Riverpod. Cache-first membuat bacaan tetap tersedia, dirty flag mencatat perubahan tertunda, tombstone menjaga penghapusan, dan conditional ACK melindungi edit saat upload. Pengujian dengan fake serta SQLite membantu memeriksa perilaku tersebut. Saya juga belajar bahwa rekomendasi AI perlu dibandingkan dengan implementasi dan bukti uji, sehingga hasil yang belum terbukti tidak boleh langsung dinyatakan lulus.